ISYS 1108 / 3474 Software Engineering Project Management

Dr Maria Spichkova

Senior Lecturer in Software Engineering

Week 4: Lectorial

What's next...

RMIT UNIVERSITY

Acknowledgement of Country

RMIT University acknowledges the people of the Woi wurrung and Boon wurrung language groups of the eastern Kulin Nation on whose unceded lands we conduct the business of the University.

RMIT University respectfully acknowledges their Ancestors and Elders, past and present.

RMIT also acknowledges the Traditional Custodians and their Ancestors of the lands and waters across Australia where we conduct our business.

Classes and Deadlines

Mid-semester break is between Weeks 5 and 6

	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Lectorials												
Workshops						Team-based assignment activities						
A1: SE Challenge				Friday: Challenge 1 10%					Friday: Challenge 2 10%		Friday: Challenge 3 10%	
A2: Team-based assignment								Due on Friday: 30%				
A3: Individual Assignment												Due on Friday: 40%

Our plan for this session

- 😊 Agile planning
- 😊 Sprint 0
- 😊 Agile time estimation
- 😊 Scrum process (Scrum meetings)

Agile Planning

Planning: When?

Rolling Wave Planning

Traditional, plan-driven project management approaches typically attempt to do more of the planning prior to the start of the project

Agile project management approaches typically defers planning decisions to “the last responsible moment”:

- You typically start with a high-level plan that is sufficient for defining at least the vision, scope, and objectives *as defined in the project charter.*
- The details of the plan and the requirements are further elaborated as the project progresses – to the level of detail is needed at that point to support whatever level of planning and estimation is required for the project
- By the ***last responsible moment***, we mean the latest point in time that a decision can be made without impacting the outcome of the overall project.

Planning strategies

Suppose you expect to complete the project in eight months, but only have clarity for the first three months:

- You would plan only for those three months.
- as the project progresses and you gain more clarity, you would then plan for the remaining months.

However, this method for planning does not exempt you from creating a list of milestones and assumptions for the entire project.

It is necessary to provide **key milestones and assumptions** as they will help stakeholders see why you are using Rolling Wave Planning and what to expect as the project progresses.

That is why you have been asked to create charter and WBS ☺

Progressive elaboration

Planning too far in advance == some amount of speculation.

Quite often, that speculation is wrong and will result in wasted effort in replanning later, might also require reworking any work that has been done based on erroneous assumptions.

Planning requirements [too far in advance] should not stop you from progressing with the implementation works in the project

Requirements should only be defined to the extent needed to support whatever decisions or action is required at that particular point in time.

Progressive elaboration

From the recommended textbook:

- At the project-level, there may be a need to understand the overall scope of the project to evaluate the resources, costs, and schedule required; however, that can typically be done on the basis of a very high-level understanding of the requirements without details.
- At the release-level, there may be a need for a little more accuracy in the estimated time and effort to complete a release. That might call for understanding the requirements in a little more detail, but that also should not require a significant amount of detail.

Finally, at the sprint level, there are typically two needs that should be satisfied prior to the start of a sprint:

- The team needs to have a reasonable estimate of the level of effort associated with the requirements to determine if they can fit into the capacity for a sprint.
- The team should not take any requirements into a sprint that have major uncertainties or issues associated with them that might block or delay the development effort.

Developing requirements in Agile project

A very important principle in Agile is **simplicity and limiting a solution to what is "just barely good enough"** to solve the problem.

There is an **optimum point**, and beyond that point, adding additional features has diminishing value.

It requires **close collaboration with the stakeholders** to find where that optimum point is.

A good way is

- to start with the simplest and most basic solution possible
- then add to it incrementally and iteratively only to the extent that it adds value to the user.

MoSCoW technique

Must have:

- Requirements labelled as “MUST” have to be included in the current delivery time box in order for it to be a success.
- If even one “MUST” requirement is not included, the project delivery should be considered a failure.

Should have:

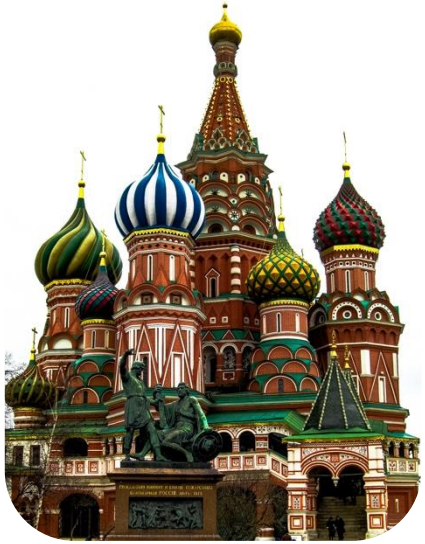
- “SHOULD” requirements are also critical to the success of the project
- Are not necessary for delivery in the current delivery time box.

Could have:

- Requirements labelled as “COULD” are often seen as nice to have.

Won’t have:

- “WON’T” requirements are either the least-critical, lowest-payback items, or not appropriate at that time.



1.Vision & Scope document: Specify the product features

2.Product Backlog: Specify PBIs on the basis of the product features. Some PBIs might be technical, but most of them are functional.

- *PBIs are about **the what***
- *PBIs represent business value (ideally, from the customers' perspective)*
- *PBI == user story / epic*
- *User story should be small enough to be completed within a sprint*

3.Sprint Backlog: Specify tasks on the basis of PBIs.

- *Sprint tasks are about **the how***
- *A task is a step by the team to create the business value (represented by PBI)*
- *A task should be small – no bigger that 1 day of work*
- *Some teams prefer to specify small enough PBIs, so that tasks are not even necessary*

Plan progressively...

You can see it as a kind of refinement...

Sprint 0

there is no such thing in the official Scrum guide ☺

It is a term that has become popular over the last few years.

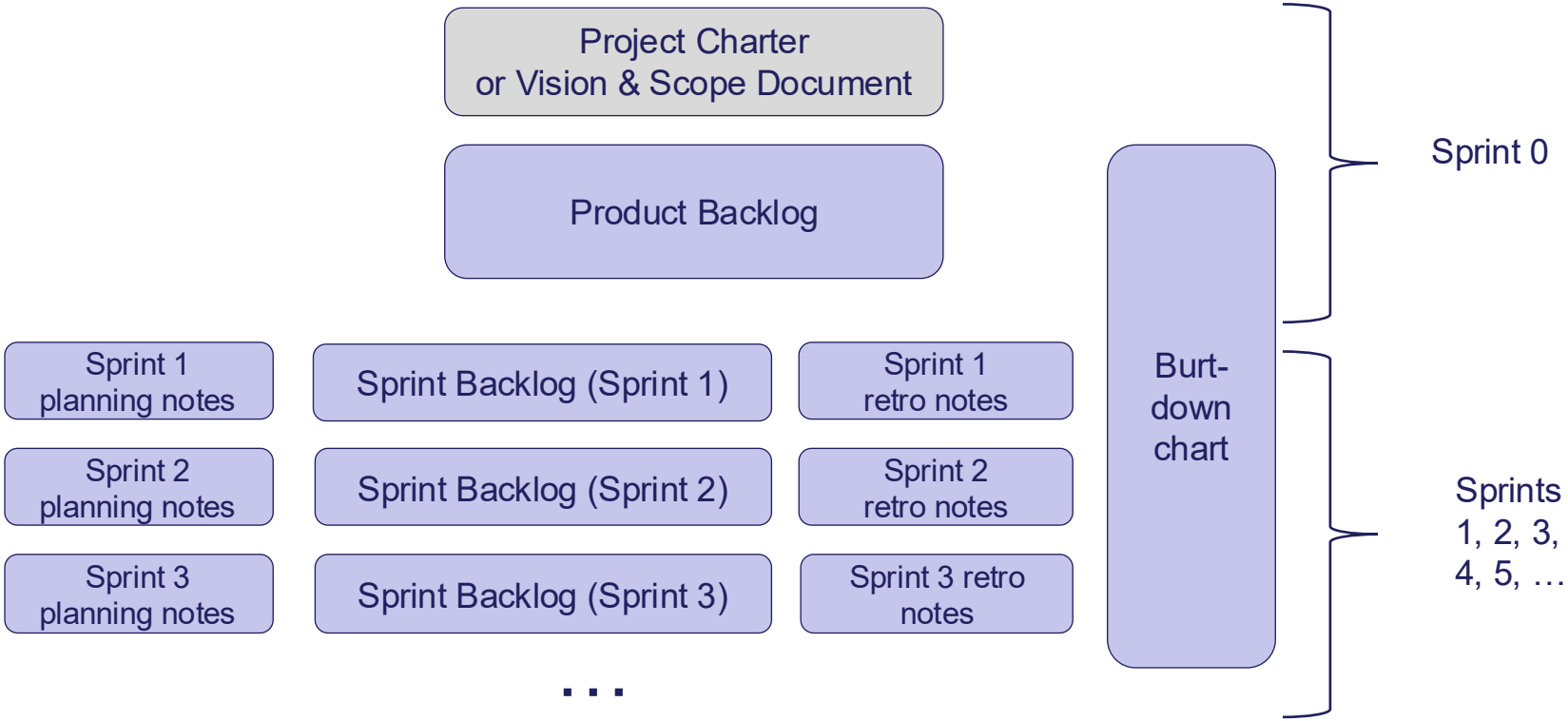
Sprint 0 == the time when the following artefacts are created:

- Vision & Scope (or Product Charter) documents,
- rough product backlog which allows creating an estimation of a product release(s).

Some Scrum Masters (Scrum purists ☺) still consider it as a bad practice: you do not have typical Scrum meetings over that period, the activities within Sprint 0 do not require burn down chart (the burn down starts from Sprint 1), etc. separate the pre-sprints activities.

The popularity of this term is growing, and the many organisations like to use it, arguing that “0” highlights the special nature of the sprint.

Project Management Artefacts in Scrum



Action time ☺

Let’s go to www.menti.com ☺

Agile time estimation

Estimation of Product Backlog items

Establishes team's velocity (how much effort a team can handle in one sprint)

Units of complexity:

- Size-category ("T-Shirt size") - *we will learn about this later on lectures*
- Story points – *this is the standard Scrum approach*
- Work days/work hours - *you know about this already* ☺

Methods of estimation:

- Expert Review
- Using a **Work Breakdown Structure (WBS)**

Agile estimation issues

There are two common problems with Agile estimation:

- **Analysis Paralysis:**
  - Spending too much time attempting to develop detailed estimates
  - Delaying making commitments until all of the information required to make decision has high level of certainty.
- **Cavalier approach:**
  - Not worrying about managing uncertainty and risk at all and just starting the project with no planning at all
  - Assuming that whatever uncertainty and risk is inherent in the project will be discovered and somehow work itself out as the project succeeds

…. So management of uncertainty is very important

Story Points

Traditional software teams give estimates in a time format: days, weeks, months.

Many agile teams, however, have transitioned to **story points**.

Story points rate the relative effort of work in a **Fibonacci format**:
0, 0.5, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144
(in many versions, a Fibonacci-like variant is used:
0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100).

Why are story points are used?

Why story points?

- ✓ Dates/hours don't account for the **non-project related work** that inevitably creeps into our days: emails, meetings, etc
- ✓ Dates/hours have an emotional attachment to them. Relative estimation removes the emotional attachment.
- ✓ Each team will estimate work on a slightly different scale, which means their velocity (measured in points) will naturally be different. This, in turn, makes it impossible to play politics using velocity as a weapon.

The relative size of story points provides some information to the product owner to assess the business value against the level of the effort

Story Point calculations

In reality it is very hard for a new agile team to start calculating story points because they have no frame of reference to compare it to.

Sometimes Product Owner/Scrum Master might make it easier for them by stating: *one story point == one person-day of work*

But, as team gets experienced such scales are not necessary – they can make their decisions based on past experience

What is the process?

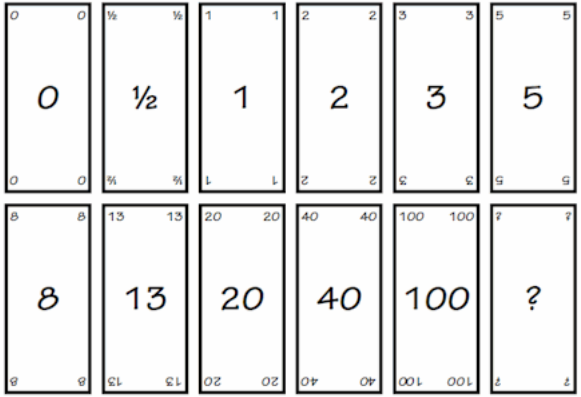
- ✓ A **voting technique** is used to estimate the level of effort associated with each story in story points
- ✓ Project team determines the **team's velocity** in story points based on number of story points completed in a sprint.
- ✓ You can then find out **how many sprints are required** =

(Total of story points)/velocity of team in story points per sprint)

Estimation techniques: Planning Poker

To begin, each team member is given a set of cards with numbers on them. The numbers are usually ordered from 0 to 21 (20) using the Fibonacci-(like) sequence: 0, 1, 2, 3, 5, 8, 13, and 21 (20).

Then each story is read aloud. After each story is presented, everybody on the team is asked to hold up the card showing the level of effort that they believe this story represents for the team.



Once all the votes are in, the team members with the lowest and highest estimates explain why they chose their scores.

Through this process, everybody on the team learns more about what's involved in estimating stories both inside and outside of their specialties, increasing knowledge sharing across the entire team.

With planning poker, the numbers are significant. A story estimated as a 2 should be about one fourth as difficult as a story estimated as an 8.

Stories estimated at 21 (20) or higher may be so large that they need to broken up into smaller stories before they can be attempted.

Estimation techniques: T-shirt sizes

Items are categorised into t-shirt sizes:

XS, S, M, L, XL, XXL.

This is a very informal technique, and can be used quickly with a large number of items.

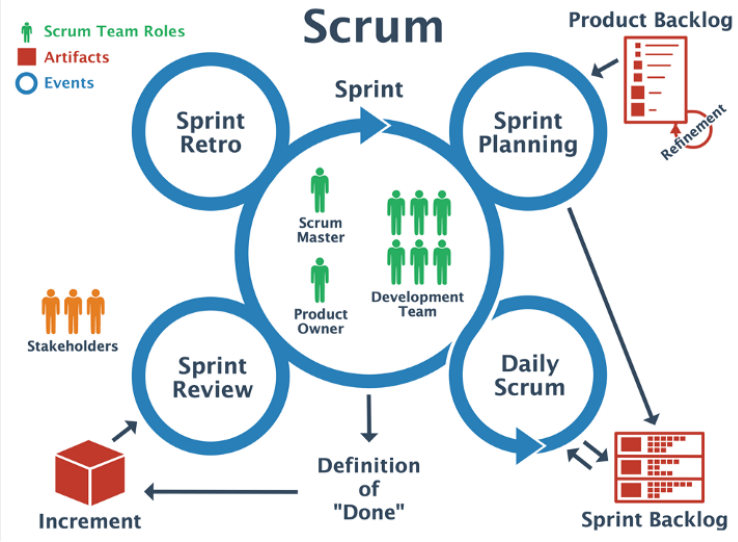
Usually, the decisions about the size are based on open, collaborative discussion, possibly with the occasional vote to break a stalemate.

The T-shirt sizes need to be converted to numerical values for the sake of tracking effort over time and charting an estimated velocity for the team.

Action time ☺

Let’s go to www.menti.com ☺

Scrum process

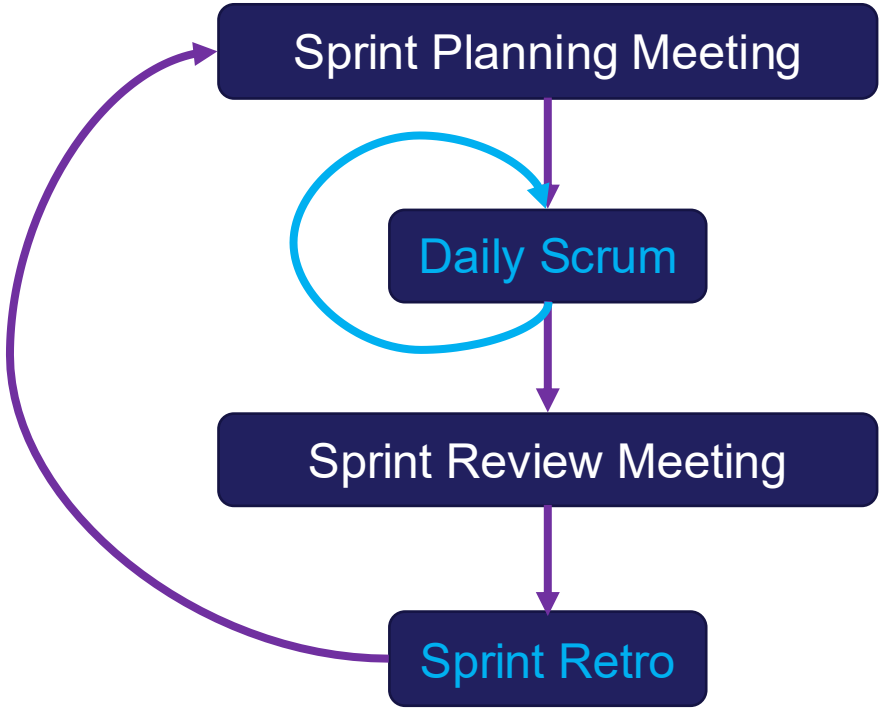


Sprint Calendar

Monday	Tuesday	Wednesday	Thursday	Friday
16	17	18 **Sprint Planning (Sprint 1)**	19 Daily standup	20 Daily standup
23 Daily standup	24 Daily standup	25 Daily standup **Backlog Refinement**	26 Daily standup	27 Daily standup
30 Daily standup	31 **Sprint Review (Sprint 1)** **Sprint Retrospective (Sprint 1)**	1 **Sprint Planning (Sprint 2)**	2 Daily standup	3 Daily standup
6 Daily standup	7 Daily standup	8 Daily standup **Backlog Refinement**	9 Daily standup	10 Daily standup

Ref: https://andybacon.com/ used for educational-purposes only

Sprints: Scrum Meetings



Backlog Refinement Meeting

Backlogs

This is something you have to complete in your assignment ☺

The **Product Backlog** represents the larger picture and lists epics and stories for a product.

A **Sprint Backlog** is concerned with the stories and tasks that are to be undertaken within a sprint.

The items in sprint backlogs come from the product backlog during sprint planning.

Agility requires learning to split large epics into user stories representing very small product features.

Backlog Refinement Meeting

also called:
- Backlog Grooming
- Backlog Maintenance
- Story Time

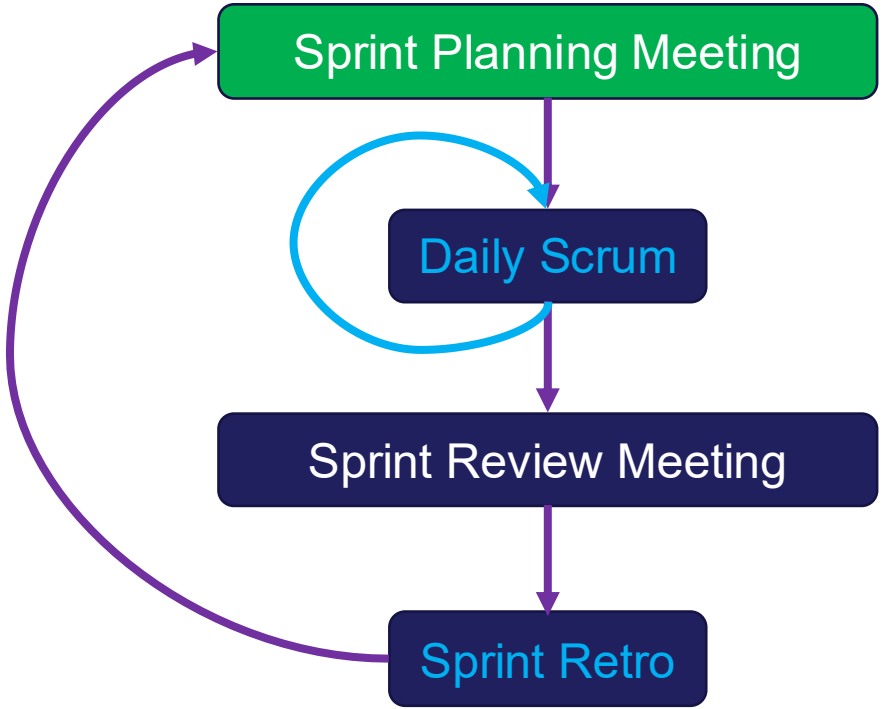
Backlog Grooming

- to prepare the Product Backlog for the next Sprint Planning Meeting.
- the team estimates the amount of effort they would expend to complete items in the Product Backlog and provides other technical information to help the Product Owner prioritize them.

Large vague items are split and clarified, considering both business and technical concerns – more valuable features to be delivered first.

Sometimes a subset of the team, in conjunction with the Product Owner and other stakeholders, will compose and split Product Backlog Items before involving the entire team in estimation.

Sprint Planning Meeting



Backlog Refinement Meeting

Sprint Planning Meeting

Sprint Planning Meeting is a meeting dedicated to the Sprint Backlog creation

Sprint Backlog

== the set of Product Backlog items selected for the Sprint

+ a plan for delivering the product Increment and realizing the Sprint Goal

The Sprint Backlog is a **forecast** by the Scrum (Development) Team

The Team modifies the Sprint Backlog throughout the Sprint.

Only the Team can change its Sprint Backlog during a Sprint.

The Sprint Backlog is a highly visible, real-time picture of the work that the Team plans to accomplish during the Sprint, and it belongs solely to the Team.

Scrum Guide 2013: “Sprint commitment” was replaced by “Sprint forecast”

So how does it all start?

1. The Product Owner **prioritises and identifies** all of the work items in the **Product Backlog (PB)**

2. The Team **estimates the workload**, focusing on the of the highest priority items

3. The Team commits a number of Product Backlog Items (PBIs) into Sprint Backlog – from the top of the PB, based on the team velocity

   Sprint Backlog ==

   a list of tasks that will be completed during a sprint

4. Each task must then be broken into sub tasks

- *Why is this necessary?*
- *Can you think of an example?*

Sprint Backlog: Template

Canvas ➔ Modules ➔ Templates, Tools, Team-allocation ➔ Useful Templates

SPRINT 1 - BACKLOG

PBI ID	PBI NAME	TASK ID	TASK	ASSIGNED TO	NOT STARTED	IN PROGRESS	COMPLETED	NOTES

Important:
If a PBI is allocated to a Sprint Backlog, you must have a User Story card for it – Acceptance Criteria should be specified

Can be replaced by a single column “Status”

To create a good Sprint Backlog...

1. All items of value that you intend to deliver at the end of sprint as a potentially shippable product
2. Involve every team member in the process
3. Discuss how every item should be implemented
4. Always have a definition of done
5. Identify all kinds of task
6. Review the sprint commitment
7. How much buffer time should be built?

◎Don’t assign tasks up front

◎Respect the time box

There are several views regarding these two points. Let’s discuss...

Sprint Planning Notes

Canvas ➔ Modules ➔ Templates, Tools, Team-allocation ➔ Useful Templates

Short notes – max. 2 pages

1. Goal

What is the aim that you are trying to achieve by running this sprint?

2. Duration of the sprint

2 weeks

3. What is the team's vision for this sprint?

Which items of the product backlog will be committed to sprint backlog and why?

What will the potentially shippable product look like in the end?

What features will it have in its working form?

4. Estimation in story points

Team to estimate story points for each of the items. Provide some sort of justification.

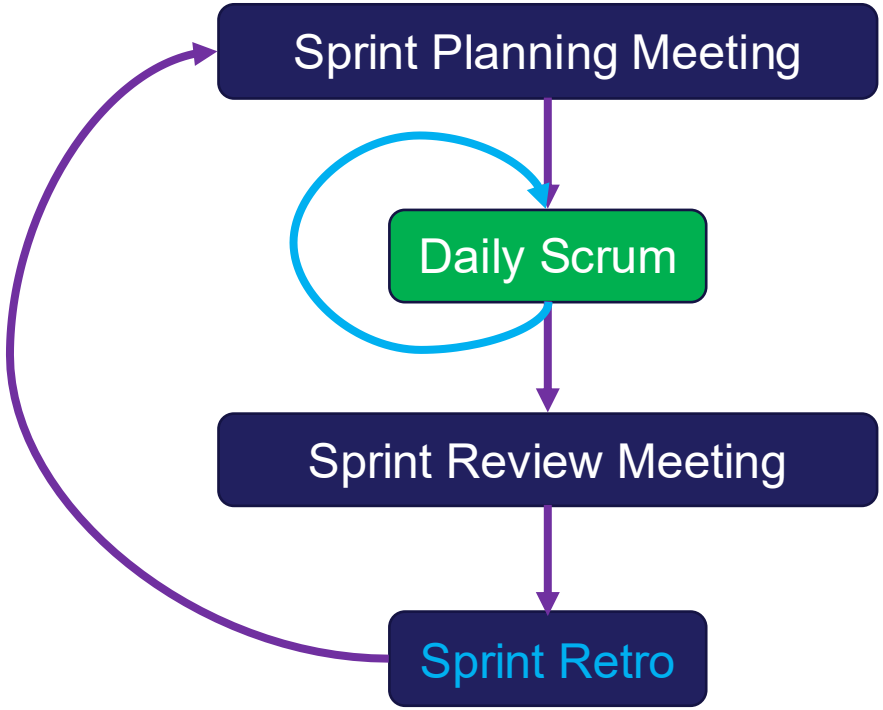
Estimation

Don’t forget that you need time not only for implementation but also for testing and integration!

Remember to make sure to…

- ✓ Use relative units (e.g., A is half as hard as B)
- ✓ Take into account team members’ skill levels, knowledge and external interruptions and unexpected technical issues
- ✓ Solicit group input on estimations
- ✓ Have everyone estimate in same units while performing *planning poker*
- ✓ Do not estimate when your team is unsure about the specifics - if not sure, ask questions (in some cases it is also acceptable to delay the item to the next sprint)
- ✓ Be realistic ☺

Daily Scrum



Backlog Refinement Meeting

Daily Stand-ups (Daily Scrum, Project Status Meetings)

Limited to 15 minutes

Are focused on the tasks at hand

Are typically held in front of a progress board

Ideally, agile teams are co-located in the same room to facilitate direct **face-to-face** communications

Are structured so that each person answers three primary questions:

1. What did you accomplish yesterday?
2. What are you going to accomplish today?
3. What obstacles are in your way?

In many situations, it isn't possible to have that level of collocation and the communications practices

- distributed teams that may not be in the same geographical location and could even be in different time zones, which is often the case with offshore development teams.

➔ you have to adjust the communications strategy to fit the team

In case of geographically distributed teams

Video conferencing might be used in lieu of face-to-face communications.

Wikis might be used for sharing of information.

Online status boards might be used in lieu of physical wall boards

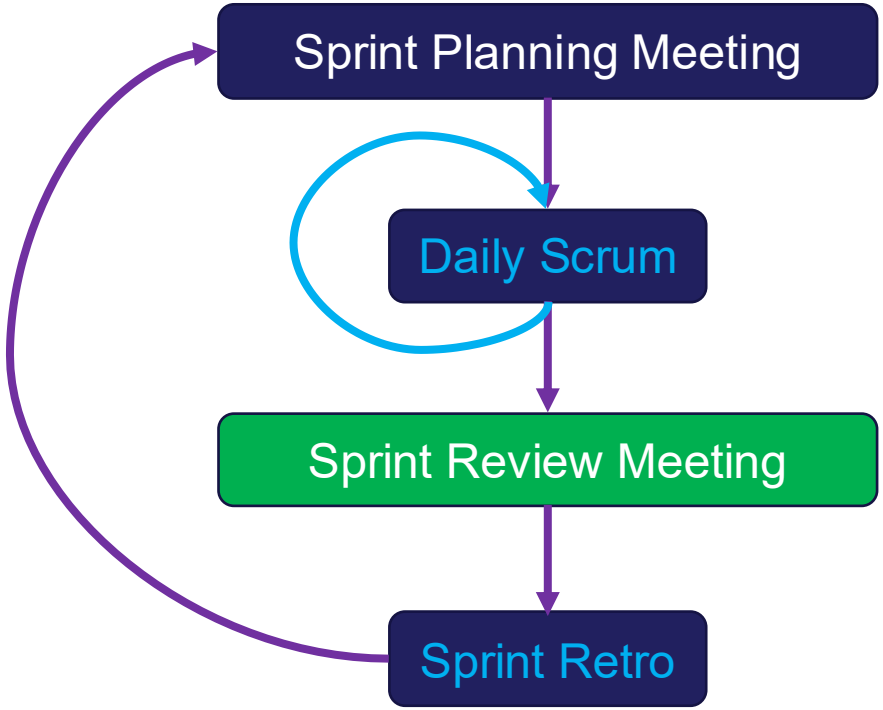
- For example, Trello or Jira

Daily standups might need to be extended or supplemented with other meetings to provide more communications

- for example, using slack.com

However, online tools are often used also by the teams that work from the same location.
Why?

Sprint Review Meeting



Backlog Refinement Meeting

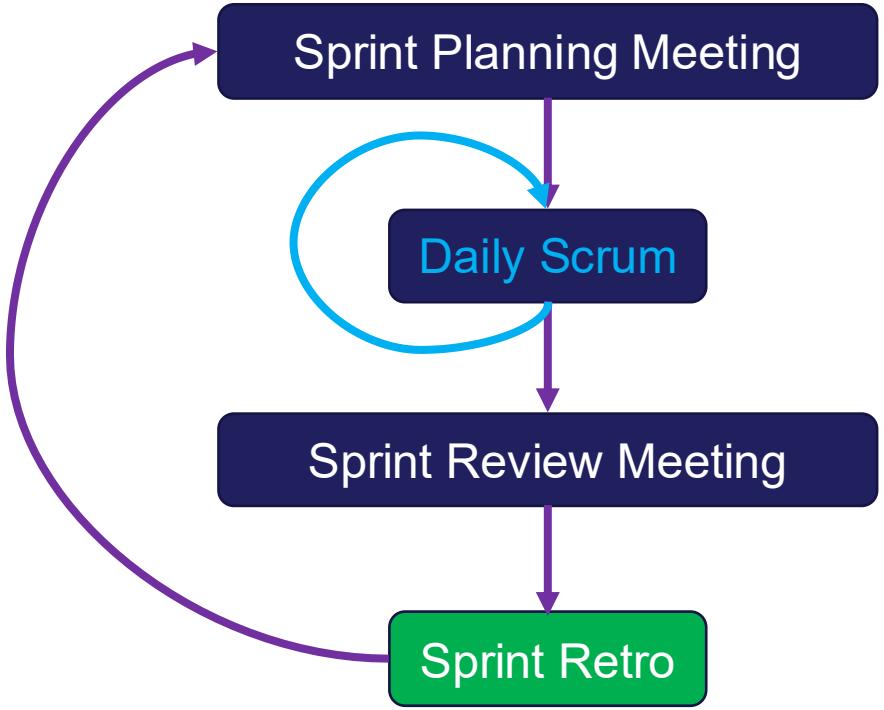
Sprint Review Meeting

Appropriate to invite external stakeholders to attend:

- It is the opportunity to inspect and adapt the product as it emerges, and iteratively refine everyone's understanding of the requirements
- Many customers need to be able to react to a piece of functioning software to discover what they will actually want. Iterative development, a *value-driven* approach, allows the creation of products that couldn't have been specified up front in a *plan-driven* approach.

1. **Demonstration (demo)**
   - Live demo not a report
   - Do not spend extra time on preparing slides, if this can be avoided
2. **Product Owner reviews the commitments made at the Sprint Planning Meeting and**
   - declares which items he/she now considers done
   - incomplete items are returned to the Product Backlog and ranked according to the Product Owner's revised priorities as candidates for future Sprints

Sprint Retro Meeting



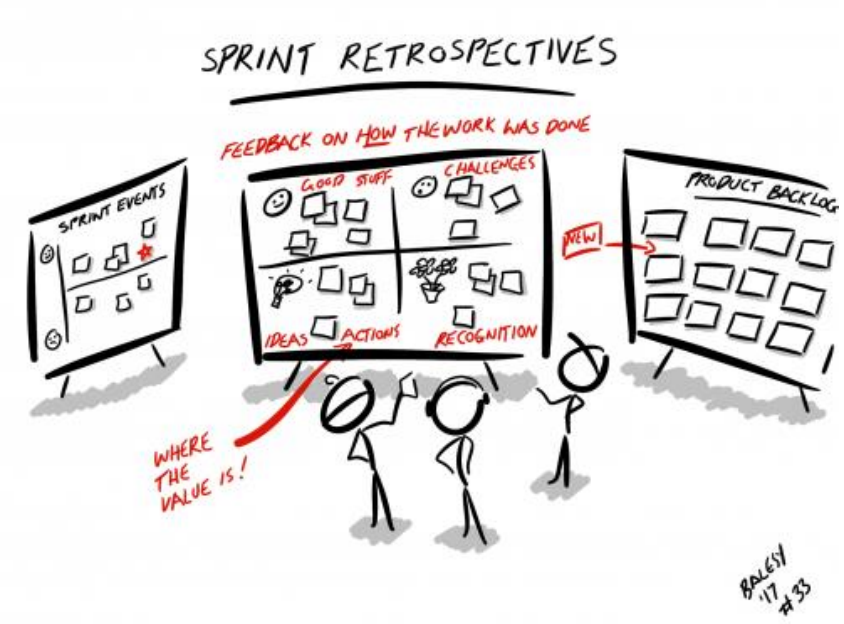
Backlog Refinement Meeting

Sprint Retrospective (Retro) Meeting

Each Sprint ends with a retrospective meeting - the team reflects on its own process:

- What went well?
- What could have gone better?
- What do we want to try next?
- What puzzles us?

They inspect their behavior and take action to adapt it for future Sprints.



An in-depth retrospective requires **an environment psychological safety** not found in most organisations.

Without safety, the retrospective discussion will either avoid the uncomfortable issues or deteriorate into blaming and hostility.

There are several solutions for this

Sprint Retro Notes

Canvas ➔ Modules ➔ Templates, Tools, Team-allocation ➔ Useful Templates

1. Things That Went Well

What went well? What the team is happy about?

2. Things That Could Have Gone Better

What could have gone better? What the team could improve?

3. Things That Surprised Us

What wasn't expected?

4. Lessons Learned

What you learned from the above points?

5. Final Thoughts

Things to Keep

Things to Change

Short notes – max. 2 pages

Sprint planning and retro notes: What we need to check?

1. Do the planning notes match to the backlog / Trello board?

2. Do the planning notes clearly specify the goal and vision for each sprint?

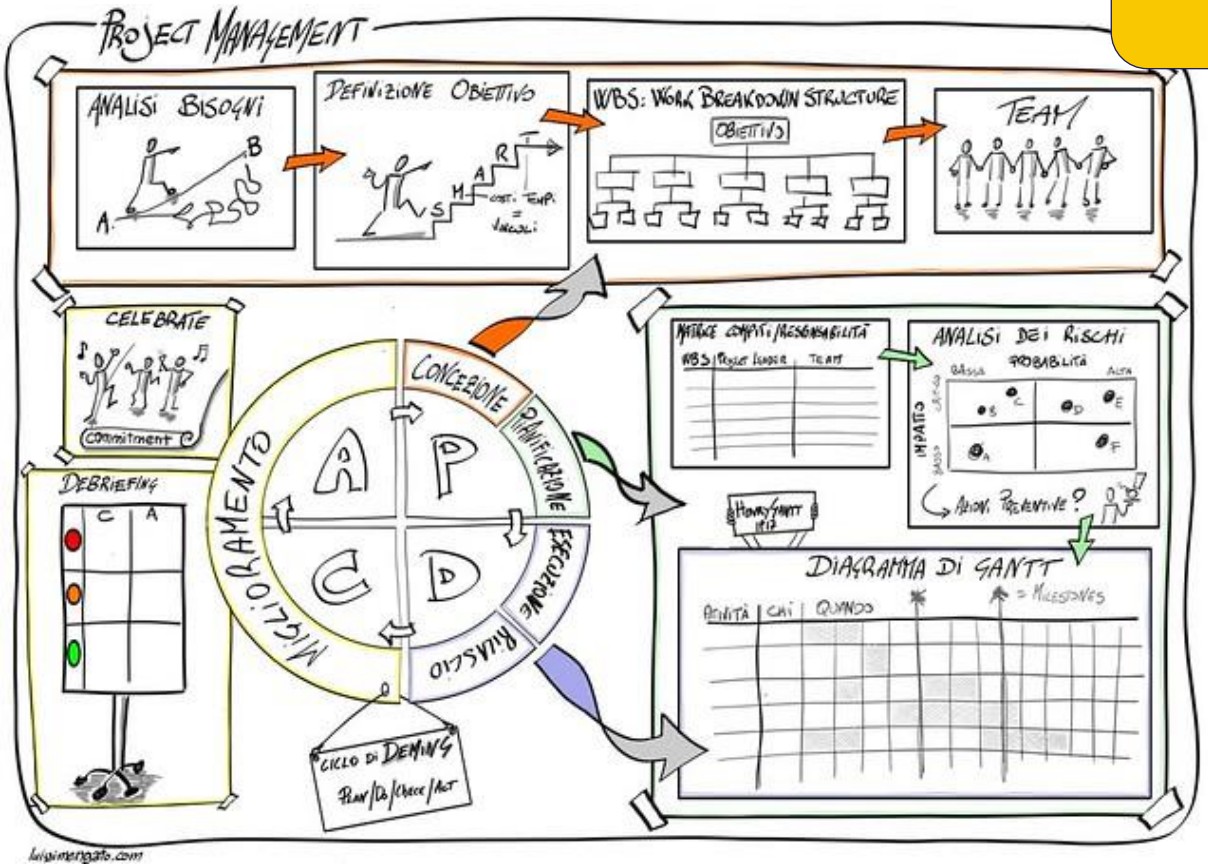
3. Do the planning notes provide a proper justification for the estimations?

4. Do the retro notes match to the backlog / Trello board?

5. Do the retro notes properly reflect the progress within the sprint and, in the case the team had any issues, the issues as well?

Action time ☺

Let’s go to www.menti.com ☺

Questions?

Essentially, all models are wrong, but some are useful.
George E.P. Box